

Dataset

- patients.csv
- doctors.csv
- insurance_providers.csv
- departments.csv
- appointments.csv
- encounters.csv
- diagnoses.csv
- medications.csv
- procedures.csv
- billing.csv

step01

create adls and upload dataset as per below mention system

containers-

- raw
- bronze
- silver
- gold
- archive

upload dataset in raw container/healthcare

kindly change storage account name and location

Complete Healthcare Analytics Project with All Datasets

ADF + Databricks (Unity Catalog) + Synapse - Step by Step

Project Overview

Datasets (10 Tables)

#	Table Name	Rows	File Name
1	patients	200	patients.csv
2	encounters	200	encounters.csv
3	diagnoses	200	diagnoses.csv
4	procedures	200	procedures.csv
5	medications	200	medications.csv
6	doctors	20	doctors.csv
7	departments	8	departments.csv
8	insurance_provider s	10	insurance_providers.cs v
9	appointments	200	appointments.csv

1
0 billing 200 billing.csv

Phase 1: Azure Infrastructure Setup (GUI)

Step 1.1: Create Resource Group

GUI Steps:

- 1. Go to [Azure Portal](#)
- 2. Click **"Create a resource"** → Search **"Resource group"**
- 3. Fill in:
 - **Subscription:** Your subscription
 - **Resource group name:** rg-healthcare-analytics-prod
 - **Region:** Central US
- 4. Click **"Review + create"** → **"Create"**

Step 1.2: Create Storage Account (ADLS Gen2)

GUI Steps:

- 1. Click **"Create a resource"** → Search **"Storage account"**
- 2. Fill in:
 - **Subscription:** Your subscription
 - **Resource group:** rg-healthcare-analytics-prod
 - **Storage account name:** healthcaredatalake789 (must be unique)
 - **Region:** Central US
 - **Performance:** Standard
 - **Redundancy:** Locally-redundant storage (LRS)
- 3. Click **"Advanced"** tab:
 - **Enable hierarchical namespace:** Checked ✓
- 4. Click **"Review + create"** → **"Create"**

Step 1.3: Create Containers

GUI Steps:

- 1. Go to your storage account → **"Containers"** (left menu)
- 2. Click **"+ Container"** for each:
 - **Name:** raw → **Public access level:** Private → **Create**
 - **Name:** bronze → **Public access level:** Private → **Create**
 - **Name:** silver → **Public access level:** Private → **Create**
 - **Name:** gold → **Public access level:** Private → **Create**

Alternative using Cloud Shell (Azure Portal > Cloud Shell icon):

```
bash
storageAccountName="healthcaredatalake789"

for container in raw bronze silver gold
do
    az storage container create \
        --name $container \
        --account-name $storageAccountName \
        --auth-mode login
done

# Verify containers
az storage container list \
    --account-name healthcaredatalake789 \
    --auth-mode login \
    --output table
```

Step 1.4: Create Databricks Workspace

GUI Steps:

- 1. Click **"Create a resource"** → Search **"Azure Databricks"**
- 2. Fill in:

- **Workspace name:** db-healthcare-analytics-prod
 - **Resource group:** rg-healthcare-analytics-prod
 - **Location:** Central US
 - **Pricing Tier:** Premium (required for Unity Catalog)
3. Click **"Review + create"** → **"Create"**

Step 1.5: Create Synapse Analytics Workspace

GUI Steps:

1. Click **"Create a resource"** → Search **"Azure Synapse Analytics"**
2. Fill in:
 - **Workspace name:** synapse-healthcare-prod
 - **Resource group:** rg-healthcare-analytics-prod
 - **Region:** Central US
 - **Data Lake Storage Gen2:** Select your storage account healthcaredatalake789
 - **File system:** gold
3. Click **"Review + create"** → **"Create"**

Phase 2: Upload Datasets to Raw Container

Step 2.1: Upload Files Using GUI

GUI Steps:

1. Go to storage account healthcaredatalake789
2. Click **"Containers"** → Click **"raw"**
3. Click **"Upload"** → **"Browse for files"**
4. Select all 10 CSV files:
 - patients.csv
 - encounters.csv
 - diagnoses.csv
 - procedures.csv
 - medications.csv
 - doctors.csv
 - departments.csv
 - insurance_providers.csv
 - appointments.csv
 - billing.csv
5. Click **"Upload"**

Step 2.2: Verify Upload

Azure CLI:

```
bash
az storage blob list \
  --container-name raw \
  --account-name healthcaredatalake789 \
  --auth-mode login \
  --output table
```

Phase 3: Databricks Unity Catalog Setup

Step 3.1: Launch Databricks Workspace

GUI Steps:

1. Go to Azure Portal → db-healthcare-analytics-prod
2. Click **"Launch Workspace"**

Step 3.2: Create Access Connector for Databricks

GUI Steps:

1. In Azure Portal → **"Create a resource"** → Search **"Access Connector for Databricks"**

2. Fill in:

- **Connector name:** adls-access-connector-prod
- **Resource group:** rg-healthcare-analytics-prod
- **Region:** Central US

3. Click **"Create"**

Step 3.3: Grant RBAC Permissions to Access Connector

GUI Steps:

1. Go to storage account healthcaredatalake789
2. Click **"Access Control (IAM)"** → **"+ Add"** → **"Add role assignment"**
3. Select role: **"Storage Blob Data Contributor"**
4. Assign access to: **"Managed identity"**
5. Select: **"Access Connector for Databricks"** → Choose adls-access-connector-prod
6. Click **"Review + assign"**

Step 3.4: Create Databricks Cluster

GUI Steps in Databricks Workspace:

1. Click **"Compute"** (left sidebar)
2. Click **"Create Cluster"**
3. Configure:
 - **Cluster name:** healthcare-cluster-prod
 - **Runtime:** 13.3 LTS (Scala 2.12, Spark 3.4.1)
 - **Unity Catalog:** Enabled (toggle ON)
 - **Node type:** Standard_DS3_v2
 - **Min workers:** 1
 - **Max workers:** 4
4. Click **"Create Cluster"**

Step 3.5: Create Storage Credential

SQL in Databricks Notebook:

```
sql
-- Create storage credential for ADLS access
CREATE STORAGE CREDENTIAL adls_credential_prod
WITH (
    AZURE_MI = '/subscriptions/YOUR_SUBSCRIPTION_ID/resourceGroups/rg-healthcare-analytics-
prod/providers/Microsoft.Databricks/accessConnectors/adls-access-connector-prod'
);

-- Verify credential
DESCRIBE STORAGE CREDENTIAL adls_credential_prod;
```

Note: Replace YOUR_SUBSCRIPTION_ID with your actual subscription ID

Step 3.6: Create External Locations

```
%sql
-- Raw container Location
CREATE EXTERNAL LOCATION raw_location_prod
URL 'abfss://raw@healthcaredatalake789.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL adls_credential_prod);

-- Bronze container Location
CREATE EXTERNAL LOCATION bronze_location_prod
URL 'abfss://bronze@healthcaredatalake789.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL adls_credential_prod);

-- Silver container Location
CREATE EXTERNAL LOCATION silver_location_prod
URL 'abfss://silver@healthcaredatalake789.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL adls_credential_prod);

-- Gold container Location
CREATE EXTERNAL LOCATION gold_location_prod
URL 'abfss://gold@healthcaredatalake789.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL adls_credential_prod);

-- Show all external Locations
```

```
SHOW EXTERNAL LOCATIONS;
```

Step 3.7: Create Catalog and Schemas

```
sql
-- Create enterprise catalog
CREATE CATALOG healthcare_catalog_prod
COMMENT "Healthcare analytics catalog"
MANAGED LOCATION 'abfss://bronze@healthcaredatalake789.dfs.core.windows.net/';

-- Create schemas for medallion layers
CREATE SCHEMA healthcare_catalog_prod.bronze
COMMENT "Raw healthcare data"
MANAGED LOCATION 'abfss://bronze@healthcaredatalake789.dfs.core.windows.net/';

CREATE SCHEMA healthcare_catalog_prod.silver
COMMENT "Cleaned healthcare data"
MANAGED LOCATION 'abfss://silver@healthcaredatalake789.dfs.core.windows.net/';

CREATE SCHEMA healthcare_catalog_prod.gold
COMMENT "Aggregated healthcare analytics"
MANAGED LOCATION 'abfss://gold@healthcaredatalake789.dfs.core.windows.net/';

-- Show catalogs
SHOW CATALOGS;

-- Show schemas
SHOW SCHEMAS IN healthcare_catalog_prod;
```

Optional One way of reading files

Read data from adls Raw container csv file

```
raw_path = "abfss://raw@adlsdatabricks4000.dfs.core.windows.net/customer.csv"

df = spark.read.option("header", True).option("inferSchema", True).csv(raw_path)

display(df)
```

Read All Datasets from Raw Container

Single Notebook: 00_read_all_datasets

```
python
# Databricks Notebook: 00_read_all_datasets
# Purpose: Read all 10 healthcare CSV files from ADLS Raw container

print("="*60)
print("READING ALL HEALTHCARE DATASETS FROM ADLS")
print("="*60)

# Storage account configuration
storage_account = "adlshealthcare3000"
raw_base_path = f"abfss://raw@{storage_account}.dfs.core.windows.net/"

# =====
# 1. Read Patients Dataset
```



```

# =====
print("\n1. Reading Patients Dataset...")
patients_path = f"{raw_base_path}patients.csv"

df_patients = spark.read.option("header", True).option("inferSchema", True).csv(patients_path)

print(f"✓ Patients records: {df_patients.count()}")
print("Schema:")
df_patients.printSchema()
print("Sample data:")
display(df_patients.limit(5))

# =====
# 2. Read Encounters Dataset
# =====
print("\n2. Reading Encounters Dataset...")
encounters_path = f"{raw_base_path}encounters.csv"

df_encounters = spark.read.option("header", True).option("inferSchema", True).csv(encounters_path)

print(f"✓ Encounters records: {df_encounters.count()}")
print("Schema:")
df_encounters.printSchema()
print("Sample data:")
display(df_encounters.limit(5))

# =====
# 3. Read Diagnoses Dataset
# =====
print("\n3. Reading Diagnoses Dataset...")
diagnoses_path = f"{raw_base_path}diagnoses.csv"

df_diagnoses = spark.read.option("header", True).option("inferSchema", True).csv(diagnoses_path)

print(f"✓ Diagnoses records: {df_diagnoses.count()}")
print("Schema:")
df_diagnoses.printSchema()
print("Sample data:")
display(df_diagnoses.limit(5))

# =====
# 4. Read Procedures Dataset
# =====
print("\n4. Reading Procedures Dataset...")
procedures_path = f"{raw_base_path}procedures.csv"

df_procedures = spark.read.option("header", True).option("inferSchema", True).csv(procedures_path)

print(f"✓ Procedures records: {df_procedures.count()}")
print("Schema:")
df_procedures.printSchema()
print("Sample data:")
display(df_procedures.limit(5))

# =====
# 5. Read Medications Dataset
# =====
print("\n5. Reading Medications Dataset...")
medications_path = f"{raw_base_path}medications.csv"

```

```

df_medications = spark.read.option("header", True).option("inferSchema",
True).csv(medications_path)

print(f"✓ Medications records: {df_medications.count()}")
print("Schema:")
df_medications.printSchema()
print("Sample data:")
display(df_medications.limit(5))

# =====
# 6. Read Doctors Dataset
# =====
print("\n6. Reading Doctors Dataset...")
doctors_path = f"{raw_base_path}doctors.csv"

df_doctors = spark.read.option("header", True).option("inferSchema", True).csv(doctors_path)

print(f"✓ Doctors records: {df_doctors.count()}")
print("Schema:")
df_doctors.printSchema()
print("Sample data:")
display(df_doctors.limit(5))

# =====
# 7. Read Departments Dataset
# =====
print("\n7. Reading Departments Dataset...")
departments_path = f"{raw_base_path}departments.csv"

df_departments = spark.read.option("header", True).option("inferSchema",
True).csv(departments_path)

print(f"✓ Departments records: {df_departments.count()}")
print("Schema:")
df_departments.printSchema()
print("Sample data:")
display(df_departments.limit(5))

# =====
# 8. Read Insurance Providers Dataset
# =====
print("\n8. Reading Insurance Providers Dataset...")
insurance_path = f"{raw_base_path}insurance_providers.csv"

df_insurance = spark.read.option("header", True).option("inferSchema", True).csv(insurance_path)

print(f"✓ Insurance Providers records: {df_insurance.count()}")
print("Schema:")
df_insurance.printSchema()
print("Sample data:")
display(df_insurance.limit(5))

# =====
# 9. Read Appointments Dataset
# =====
print("\n9. Reading Appointments Dataset...")
appointments_path = f"{raw_base_path}appointments.csv"

df_appointments = spark.read.option("header", True).option("inferSchema",
True).csv(appointments_path)

```

```

print(f"✓ Appointments records: {df_appointments.count()}")
print("Schema:")
df_appointments.printSchema()
print("Sample data:")
display(df_appointments.limit(5))

# =====
# 10. Read Billing Dataset
# =====
print("\n10. Reading Billing Dataset...")
billing_path = f"{raw_base_path}billing.csv"

df_billing = spark.read.option("header", True).option("inferSchema", True).csv(billing_path)

print(f"✓ Billing records: {df_billing.count()}")
print("Schema:")
df_billing.printSchema()
print("Sample data:")
display(df_billing.limit(5))

# =====
# Summary Report
# =====
print("\n" + "="*60)
print("DATASET SUMMARY REPORT")
print("="*60)

summary_data = [
    ("Patients", df_patients.count(), len(df_patients.columns)),
    ("Encounters", df_encounters.count(), len(df_encounters.columns)),
    ("Diagnoses", df_diagnoses.count(), len(df_diagnoses.columns)),
    ("Procedures", df_procedures.count(), len(df_procedures.columns)),
    ("Medications", df_medications.count(), len(df_medications.columns)),
    ("Doctors", df_doctors.count(), len(df_doctors.columns)),
    ("Departments", df_departments.count(), len(df_departments.columns)),
    ("Insurance Providers", df_insurance.count(), len(df_insurance.columns)),
    ("Appointments", df_appointments.count(), len(df_appointments.columns)),
    ("Billing", df_billing.count(), len(df_billing.columns))
]

from pyspark.sql import Row
summary_df = spark.createDataFrame(
    [Row(Table=row[0], Record_Count=row[1], Columns=row[2]) for row in summary_data]
)
display(summary_df)

print("\n All 10 datasets loaded successfully!")

```

Read with Explicit Schemas (Better Performance)

python

Databricks Notebook: Read with explicit schemas for better performance

```

from pyspark.sql.functions import *
from pyspark.sql.types import *
from datetime import datetime

```



```

print("="*60)
print("READING HEALTHCARE DATASETS WITH EXPLICIT SCHEMAS")
print("="*60)

storage_account = "adlshealthcare3000"
raw_base_path = f"abfss://raw@{storage_account}.dfs.core.windows.net/"

# =====
# Define schemas for all tables
# =====

patients_schema = StructType([
    StructField("patient_id", StringType(), True),
    StructField("first_name", StringType(), True),
    StructField("last_name", StringType(), True),
    StructField("date_of_birth", DateType(), True),
    StructField("gender", StringType(), True),
    StructField("marital_status", StringType(), True),
    StructField("race", StringType(), True),
    StructField("ethnicity", StringType(), True),
    StructField("language", StringType(), True),
    StructField("address", StringType(), True),
    StructField("city", StringType(), True),
    StructField("state", StringType(), True),
    StructField("zip_code", StringType(), True),
    StructField("phone_number", StringType(), True),
    StructField("email", StringType(), True),
    StructField("emergency_contact", StringType(), True),
    StructField("primary_physician_id", StringType(), True),
    StructField("insurance_provider_id", StringType(), True),
    StructField("admission_date", DateType(), True),
    StructField("discharge_date", DateType(), True)
])

encounters_schema = StructType([
    StructField("encounter_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("doctor_id", StringType(), True),
    StructField("department_id", StringType(), True),
    StructField("encounter_type", StringType(), True),
    StructField("encounter_date", DateType(), True),
    StructField("admission_source", StringType(), True),
    StructField("discharge_status", StringType(), True)
])

diagnoses_schema = StructType([
    StructField("diagnosis_id", StringType(), True),
    StructField("encounter_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("diagnosis_code", StringType(), True),
    StructField("diagnosis_description", StringType(), True),
    StructField("diagnosis_date", DateType(), True),
    StructField("severity", StringType(), True)
])

procedures_schema = StructType([
    StructField("procedure_id", StringType(), True),
    StructField("encounter_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("doctor_id", StringType(), True),
    StructField("procedure_name", StringType(), True),
    StructField("procedure_date", DateType(), True),
    StructField("cost", DoubleType(), True)
])

medications_schema = StructType([
    StructField("medication_id", StringType(), True),

```

```

    StructField("patient_id", StringType(), True),
    StructField("doctor_id", StringType(), True),
    StructField("medicine_name", StringType(), True),
    StructField("dosage", StringType(), True),
    StructField("frequency", StringType(), True),
    StructField("start_date", DateType(), True),
    StructField("end_date", DateType(), True)
])

doctors_schema = StructType([
    StructField("doctor_id", StringType(), True),
    StructField("doctor_name", StringType(), True),
    StructField("specialty", StringType(), True),
    StructField("department_id", StringType(), True),
    StructField("phone_number", StringType(), True),
    StructField("email", StringType(), True),
    StructField("hire_date", DateType(), True)
])

departments_schema = StructType([
    StructField("department_id", StringType(), True),
    StructField("department_name", StringType(), True),
    StructField("floor_number", IntegerType(), True),
    StructField("manager_name", StringType(), True)
])

insurance_schema = StructType([
    StructField("insurance_provider_id", StringType(), True),
    StructField("provider_name", StringType(), True),
    StructField("plan_type", StringType(), True),
    StructField("coverage_percent", IntegerType(), True),
    StructField("contact_number", StringType(), True)
])

appointments_schema = StructType([
    StructField("appointment_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("doctor_id", StringType(), True),
    StructField("appointment_date", DateType(), True),
    StructField("appointment_time", StringType(), True),
    StructField("status", StringType(), True),
    StructField("reason", StringType(), True)
])

billing_schema = StructType([
    StructField("bill_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("encounter_id", StringType(), True),
    StructField("insurance_provider_id", StringType(), True),
    StructField("billing_date", DateType(), True),
    StructField("total_amount", DoubleType(), True),
    StructField("insurance_paid", DoubleType(), True),
    StructField("patient_paid", DoubleType(), True),
    StructField("payment_status", StringType(), True)
])

# =====
# Read all datasets with schemas
# =====

print("\nReading datasets with explicit schemas...\n")

# Patients
patients_path = f"{raw_base_path}patients.csv"
df_patients = spark.read.option("header", True).schema(patients_schema).csv(patients_path)
print(f"✓ Patients: {df_patients.count()} records")

# Encounters

```

```

encounters_path = f"{raw_base_path}encounters.csv"
df_encounters = spark.read.option("header", True).schema(encounters_schema).csv(encounters_path)
print(f"✓ Encounters: {df_encounters.count()} records")

# Diagnoses
diagnoses_path = f"{raw_base_path}diagnoses.csv"
df_diagnoses = spark.read.option("header", True).schema(diagnoses_schema).csv(diagnoses_path)
print(f"✓ Diagnoses: {df_diagnoses.count()} records")

# Procedures
procedures_path = f"{raw_base_path}procedures.csv"
df_procedures = spark.read.option("header", True).schema(procedures_schema).csv(procedures_path)
print(f"✓ Procedures: {df_procedures.count()} records")

# Medications
medications_path = f"{raw_base_path}medications.csv"
df_medications = spark.read.option("header", True).schema(medications_schema).csv(medications_path)
print(f"✓ Medications: {df_medications.count()} records")

# Doctors
doctors_path = f"{raw_base_path}doctors.csv"
df_doctors = spark.read.option("header", True).schema(doctors_schema).csv(doctors_path)
print(f"✓ Doctors: {df_doctors.count()} records")

# Departments
departments_path = f"{raw_base_path}departments.csv"
df_departments = spark.read.option("header", True).schema(departments_schema).csv(departments_path)
print(f"✓ Departments: {df_departments.count()} records")

# Insurance Providers
insurance_path = f"{raw_base_path}insurance_providers.csv"
df_insurance = spark.read.option("header", True).schema(insurance_schema).csv(insurance_path)
print(f"✓ Insurance Providers: {df_insurance.count()} records")

# Appointments
appointments_path = f"{raw_base_path}appointments.csv"
df_appointments = spark.read.option("header", True).schema(appointments_schema).csv(appointments_path)
print(f"✓ Appointments: {df_appointments.count()} records")

# Billing
billing_path = f"{raw_base_path}billing.csv"
df_billing = spark.read.option("header", True).schema(billing_schema).csv(billing_path)
print(f"✓ Billing: {df_billing.count()} records")

# =====
# Display sample from each table
# =====

print("\n" + "="*60)
print("SAMPLE DATA FROM EACH TABLE")
print("="*60)

print("\n1. PATIENTS SAMPLE:")
display(df_patients.limit(5))

print("\n2. ENCOUNTERS SAMPLE:")
display(df_encounters.limit(5))

print("\n3. DIAGNOSES SAMPLE:")
display(df_diagnoses.limit(5))

print("\n4. PROCEDURES SAMPLE:")
display(df_procedures.limit(5))

print("\n5. MEDICATIONS SAMPLE:")
display(df_medications.limit(5))

```



```

print("\n6. DOCTORS SAMPLE:")
display(df_doctors.limit(5))

print("\n7. DEPARTMENTS SAMPLE:")
display(df_departments.limit(5))

print("\n8. INSURANCE PROVIDERS SAMPLE:")
display(df_insurance.limit(5))

print("\n9. APPOINTMENTS SAMPLE:")
display(df_appointments.limit(5))

print("\n10. BILLING SAMPLE:")
display(df_billing.limit(5))

print("\n  All 10 datasets loaded successfully with explicit schemas!")

```

Create delta table in adls

Optional

```

%sql
CREATE TABLE healthcare_catalog_prod.bronze.bronze_customer
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/procedures.csv/';

```

Complete Healthcare Analytics Project - Create All Bronze Delta Tables in Unity Catalog

Step-by-Step SQL Commands to Create All Bronze Delta Tables

Step 1: Write DataFrames to Bronze Container (Python)

Notebook: 01_create_bronze_delta_tables

python

```

# Databricks Notebook: 01_create_bronze_delta_tables
# Purpose: Write all healthcare datasets as Delta tables to Bronze container

```

```

from pyspark.sql.functions import input_file_name, current_timestamp, lit
from pyspark.sql.types import *
from datetime import datetime

```

```

print("="*60)
print("CREATING BRONZE DELTA TABLES FOR ALL DATASETS")
print("="*60)

```

```

# Storage account configuration
storage_account = "adlshealthcare3000"
raw_base_path = f"abfss://raw@{storage_account}.dfs.core.windows.net/"
bronze_base_path = f"abfss://bronze@{storage_account}.dfs.core.windows.net/"

```

```

# =====
# Define schemas for all tables
# =====

```

```
patients_schema = StructType([
    StructField("patient_id", StringType(), True),
    StructField("first_name", StringType(), True),
    StructField("last_name", StringType(), True),
    StructField("date_of_birth", DateType(), True),
    StructField("gender", StringType(), True),
    StructField("marital_status", StringType(), True),
    StructField("race", StringType(), True),
    StructField("ethnicity", StringType(), True),
    StructField("language", StringType(), True),
    StructField("address", StringType(), True),
    StructField("city", StringType(), True),
    StructField("state", StringType(), True),
    StructField("zip_code", StringType(), True),
    StructField("phone_number", StringType(), True),
    StructField("email", StringType(), True),
    StructField("emergency_contact", StringType(), True),
    StructField("primary_physician_id", StringType(), True),
    StructField("insurance_provider_id", StringType(), True),
    StructField("admission_date", DateType(), True),
    StructField("discharge_date", DateType(), True)
])
```

```
encounters_schema = StructType([
    StructField("encounter_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("doctor_id", StringType(), True),
    StructField("department_id", StringType(), True),
    StructField("encounter_type", StringType(), True),
    StructField("encounter_date", DateType(), True),
    StructField("admission_source", StringType(), True),
    StructField("discharge_status", StringType(), True)
])
```

```
diagnoses_schema = StructType([
    StructField("diagnosis_id", StringType(), True),
    StructField("encounter_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("diagnosis_code", StringType(), True),
    StructField("diagnosis_description", StringType(), True),
    StructField("diagnosis_date", DateType(), True),
    StructField("severity", StringType(), True)
])
```

```
procedures_schema = StructType([
    StructField("procedure_id", StringType(), True),
    StructField("encounter_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("doctor_id", StringType(), True),
    StructField("procedure_name", StringType(), True),
    StructField("procedure_date", DateType(), True),
    StructField("cost", DoubleType(), True)
])
```

```
medications_schema = StructType([
    StructField("medication_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("doctor_id", StringType(), True),
    StructField("medicine_name", StringType(), True),
    StructField("dosage", StringType(), True),
    StructField("frequency", StringType(), True),
])
```



```

        StructField("start_date", DateType(), True),
        StructField("end_date", DateType(), True)
    ])

doctors_schema = StructType([
    StructField("doctor_id", StringType(), True),
    StructField("doctor_name", StringType(), True),
    StructField("specialty", StringType(), True),
    StructField("department_id", StringType(), True),
    StructField("phone_number", StringType(), True),
    StructField("email", StringType(), True),
    StructField("hire_date", DateType(), True)
])

departments_schema = StructType([
    StructField("department_id", StringType(), True),
    StructField("department_name", StringType(), True),
    StructField("floor_number", IntegerType(), True),
    StructField("manager_name", StringType(), True)
])

insurance_schema = StructType([
    StructField("insurance_provider_id", StringType(), True),
    StructField("provider_name", StringType(), True),
    StructField("plan_type", StringType(), True),
    StructField("coverage_percent", IntegerType(), True),
    StructField("contact_number", StringType(), True)
])

appointments_schema = StructType([
    StructField("appointment_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("doctor_id", StringType(), True),
    StructField("appointment_date", DateType(), True),
    StructField("appointment_time", StringType(), True),
    StructField("status", StringType(), True),
    StructField("reason", StringType(), True)
])

billing_schema = StructType([
    StructField("bill_id", StringType(), True),
    StructField("patient_id", StringType(), True),
    StructField("encounter_id", StringType(), True),
    StructField("insurance_provider_id", StringType(), True),
    StructField("billing_date", DateType(), True),
    StructField("total_amount", DoubleType(), True),
    StructField("insurance_paid", DoubleType(), True),
    StructField("patient_paid", DoubleType(), True),
    StructField("payment_status", StringType(), True)
])

# =====
# Read and Write each table as Delta
# =====

print("\n2. Reading raw CSV files and writing as Delta tables...\n")

# 1. Patients Table
print("Processing Patients...")
df_patients = spark.read.option("header",
True).schema(patients_schema).csv(f"{raw_base_path}patients.csv")

```

```

df_patients_with_metadata = df_patients \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("patients.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_patients_path = f"{bronze_base_path}patients/"
df_patients_with_metadata.write.format("delta").mode("overwrite").save(bronze_patients_path)
print(f"✓ Patients Delta table created at: {bronze_patients_path}")

# 2. Encounters Table
print("Processing Encounters...")
df_encounters = spark.read.option("header",
True).schema(encounters_schema).csv(f"{raw_base_path}encounters.csv")
df_encounters_with_metadata = df_encounters \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("encounters.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_encounters_path = f"{bronze_base_path}encounters/"
df_encounters_with_metadata.write.format("delta").mode("overwrite").save(bronze_encounters_path)
print(f"✓ Encounters Delta table created at: {bronze_encounters_path}")

# 3. Diagnoses Table
print("Processing Diagnoses...")
df_diagnoses = spark.read.option("header",
True).schema(diagnoses_schema).csv(f"{raw_base_path}diagnoses.csv")
df_diagnoses_with_metadata = df_diagnoses \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("diagnoses.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_diagnoses_path = f"{bronze_base_path}diagnoses/"
df_diagnoses_with_metadata.write.format("delta").mode("overwrite").save(bronze_diagnoses_path)
print(f"✓ Diagnoses Delta table created at: {bronze_diagnoses_path}")

# 4. Procedures Table
print("Processing Procedures...")
df_procedures = spark.read.option("header",
True).schema(procedures_schema).csv(f"{raw_base_path}procedures.csv")
df_procedures_with_metadata = df_procedures \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("procedures.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_procedures_path = f"{bronze_base_path}procedures/"
df_procedures_with_metadata.write.format("delta").mode("overwrite").save(bronze_procedures_path)
print(f"✓ Procedures Delta table created at: {bronze_procedures_path}")

# 5. Medications Table
print("Processing Medications...")
df_medications = spark.read.option("header",
True).schema(medications_schema).csv(f"{raw_base_path}medications.csv")
df_medications_with_metadata = df_medications \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("medications.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_medications_path = f"{bronze_base_path}medications/"
df_medications_with_metadata.write.format("delta").mode("overwrite").save(bronze_medications_path)
print(f"✓ Medications Delta table created at: {bronze_medications_path}")

# 6. Doctors Table
print("Processing Doctors...")
df_doctors = spark.read.option("header",
True).schema(doctors_schema).csv(f"{raw_base_path}doctors.csv")

```



```

df_doctors_with_metadata = df_doctors \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("doctors.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_doctors_path = f"{bronze_base_path}doctors/"
df_doctors_with_metadata.write.format("delta").mode("overwrite").save(bronze_doctors_path)
print(f"✓ Doctors Delta table created at: {bronze_doctors_path}")

# 7. Departments Table
print("Processing Departments...")
df_departments = spark.read.option("header",
True).schema(departments_schema).csv(f"{raw_base_path}departments.csv")
df_departments_with_metadata = df_departments \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("departments.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_departments_path = f"{bronze_base_path}departments/"
df_departments_with_metadata.write.format("delta").mode("overwrite").save(bronze_departments_path)
print(f"✓ Departments Delta table created at: {bronze_departments_path}")

# 8. Insurance Providers Table
print("Processing Insurance Providers...")
df_insurance = spark.read.option("header",
True).schema(insurance_schema).csv(f"{raw_base_path}insurance_providers.csv")
df_insurance_with_metadata = df_insurance \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("insurance_providers.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_insurance_path = f"{bronze_base_path}insurance_providers/"
df_insurance_with_metadata.write.format("delta").mode("overwrite").save(bronze_insurance_path)
print(f"✓ Insurance Providers Delta table created at: {bronze_insurance_path}")

# 9. Appointments Table
print("Processing Appointments...")
df_appointments = spark.read.option("header",
True).schema(appointments_schema).csv(f"{raw_base_path}appointments.csv")
df_appointments_with_metadata = df_appointments \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("appointments.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_appointments_path = f"{bronze_base_path}appointments/"
df_appointments_with_metadata.write.format("delta").mode("overwrite").save(bronze_appointments_path)
print(f"✓ Appointments Delta table created at: {bronze_appointments_path}")

# 10. Billing Table
print("Processing Billing...")
df_billing = spark.read.option("header",
True).schema(billing_schema).csv(f"{raw_base_path}billing.csv")
df_billing_with_metadata = df_billing \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("billing.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_billing_path = f"{bronze_base_path}billing/"
df_billing_with_metadata.write.format("delta").mode("overwrite").save(bronze_billing_path)
print(f"✓ Billing Delta table created at: {bronze_billing_path}")

print("\n" + "="*60)
print("    ALL BRONZE DELTA TABLES CREATED SUCCESSFULLY!")
print("="*60)

```

optional

Create delta table in adls

```
%sql
CREATE TABLE enterprise_catalog.bronze.bronze_customer
USING DELTA
LOCATION 'abfss://bronze@adlsdatabricks4000.dfs.core.windows.net/customer/';
```

Use this to create multiple delta table

```
%sql
-- =====
-- CREATE ALL BRONZE TABLES IN UNITY CATALOG
-- =====

-- Switch to healthcare catalog
USE CATALOG healthcare_catalog_prod;
USE SCHEMA bronze;

-- 1. Patients Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_patients
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/patients/';

-- 2. Encounters Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_encounters
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/encounters/';

-- 3. Diagnoses Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_diagnoses
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/diagnoses/';

-- 4. Procedures Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_procedures
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/procedures/';

-- 5. Medications Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_medications
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/medications/';

-- 6. Doctors Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_doctors
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/doctors/';

-- 7. Departments Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_departments
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/departments/';

-- 8. Insurance Providers Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_insurance_providers
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/insurance_providers/';

-- 9. Appointments Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_appointments
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/appointments/';
```



```

-- 10. Billing Bronze Table
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.bronze.bronze_billing
USING DELTA
LOCATION 'abfss://bronze@adlshealthcare3000.dfs.core.windows.net/billing/';

-- =====
-- VERIFY ALL TABLES CREATED
-- =====

-- Show all tables in bronze schema
SHOW TABLES IN healthcare_catalog_prod.bronze;

-- Verify each table by checking record counts
SELECT 'bronze_patients' as table_name, COUNT(*) as record_count FROM
healthcare_catalog_prod.bronze.bronze_patients
UNION ALL
SELECT 'bronze_encounters', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_encounters
UNION ALL
SELECT 'bronze_diagnoses', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_diagnoses
UNION ALL
SELECT 'bronze_procedures', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_procedures
UNION ALL
SELECT 'bronze_medications', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_medications
UNION ALL
SELECT 'bronze_doctors', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_doctors
UNION ALL
SELECT 'bronze_departments', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_departments
UNION ALL
SELECT 'bronze_insurance_providers', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_insurance_providers
UNION ALL
SELECT 'bronze_appointments', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_appointments
UNION ALL
SELECT 'bronze_billing', COUNT(*) FROM healthcare_catalog_prod.bronze.bronze_billing;

```

```

# Databricks Notebook: 02_silver_gold_transformations
# Purpose: Transform Bronze data to Silver and Gold layers, save as Delta tables in ADLS

from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql.window import Window
from datetime import datetime

print("="*70)
print("SILVER & GOLD LAYER TRANSFORMATIONS - ALL DATASETS")
print("="*70)

# Storage account configuration
storage_account = "adlshealthcare3000"
bronze_base_path = f"abfss://bronze@{storage_account}.dfs.core.windows.net/"
silver_base_path = f"abfss://silver@{storage_account}.dfs.core.windows.net/"
gold_base_path = f"abfss://gold@{storage_account}.dfs.core.windows.net/"

print(f"Bronze path: {bronze_base_path}")
print(f"Silver path: {silver_base_path}")
print(f"Gold path: {gold_base_path}")

# =====
# SECTION 1: SILVER LAYER TRANSFORMATIONS
# =====
print("\n" + "="*60)
print("SECTION 1: SILVER LAYER TRANSFORMATIONS")
print("="*60)

# =====
# 1.1 PATIENTS TABLE - Silver Layer
# =====
print("\n1.1 Processing Patients Data (Bronze → Silver)...")

# Read from Bronze
bronze_patients_path = f"{bronze_base_path}patients/"
df_patients_bronze = spark.read.format("delta").load(bronze_patients_path)
print(f"Bronze records: {df_patients_bronze.count()}")

# Transform to Silver
df_patients_silver = df_patients_bronze \
    .dropDuplicates(["patient_id"]) \
    .filter(col("patient_id").isNotNull()) \
    .withColumn("age",
        floor(datediff(current_date(), col("date_of_birth")) / 365.25)
    ) \
    .withColumn("age_group",
        when(col("age") < 18, "Pediatric")
        .when(col("age") < 40, "Young Adult")
        .when(col("age") < 65, "Adult")
        .otherwise("Geriatric")
    ) \
    .withColumn("full_name",
        concat(initcap(col("first_name")), lit(" "), initcap(col("last_name")))
    ) \

```

```

        .withColumn("gender_standardized",
            when(upper(col("gender")).isin("M", "MALE"), "Male")
            .when(upper(col("gender")).isin("F", "FEMALE"), "Female")
            .otherwise("Other")
        ) \
        .withColumn("email_domain", regexp_extract(col("email"), "@(.+)", 1)) \
        .withColumn("city_formatted", initcap(trim(col("city")))) \
        .withColumn("state_upper", upper(trim(col("state")))) \
        .withColumn("length_of_stay_days",
            datediff(col("discharge_date"), col("admission_date"))
        ) \
        .withColumn("silver_processing_timestamp", current_timestamp()) \
        .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
        .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_patients_silver.count()}")
print("  Sample data:")
display(df_patients_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_patients_path = f"{silver_base_path}patients_silver/"
df_patients_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_patients_path)
print(f"  ✓ Saved to: {silver_patients_path}")

# =====
# 1.2 ENCOUNTERS TABLE - Silver Layer
# =====
print("\n1.2 Processing Encounters Data (Bronze → Silver)...")

# Read from Bronze
bronze_encounters_path = f"{bronze_base_path}encounters/"
df_encounters_bronze = spark.read.format("delta").load(bronze_encounters_path)
print(f"  Bronze records: {df_encounters_bronze.count()}")

# Transform to Silver
df_encounters_silver = df_encounters_bronze \
    .dropDuplicates(["encounter_id"]) \
    .filter(col("encounter_id").isNotNull()) \
    .withColumn("encounter_year", year(col("encounter_date"))) \
    .withColumn("encounter_month", month(col("encounter_date"))) \
    .withColumn("encounter_day", dayofmonth(col("encounter_date"))) \
    .withColumn("encounter_quarter", quarter(col("encounter_date"))) \
    .withColumn("encounter_week", weekofyear(col("encounter_date"))) \
    .withColumn("encounter_dayofweek", dayofweek(col("encounter_date"))) \
    .withColumn("is_emergency", col("encounter_type") == "Emergency") \
    .withColumn("is_inpatient", col("encounter_type") == "Inpatient") \
    .withColumn("is_outpatient", col("encounter_type") == "Outpatient") \
    .withColumn("admission_source_clean",
        when(col("admission_source").isNull(), "Unknown")
        .otherwise(initcap(trim(col("admission_source"))))
    ) \
    .withColumn("discharge_status_clean",

```



```

        when(col("discharge_status").isNull(), "Unknown")
        .otherwise(initcap(trim(col("discharge_status"))))
    ) \
    .withColumn("season",
        when(col("encounter_month").isin(12, 1, 2), "Winter")
        .when(col("encounter_month").isin(3, 4, 5), "Spring")
        .when(col("encounter_month").isin(6, 7, 8), "Summer")
        .otherwise("Fall")
    ) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f" Silver records: {df_encounters_silver.count()}")
print(" Sample data:")
display(df_encounters_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_encounters_path = f"{silver_base_path}encounters_silver/"
df_encounters_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_encounters_path)
print(f" ✓ Saved to: {silver_encounters_path}")

# =====
# 1.3 DIAGNOSES TABLE - Silver Layer
# =====
print("\n1.3 Processing Diagnoses Data (Bronze → Silver)...")

# Read from Bronze
bronze_diagnoses_path = f"{bronze_base_path}diagnoses/"
df_diagnoses_bronze = spark.read.format("delta").load(bronze_diagnoses_path)
print(f" Bronze records: {df_diagnoses_bronze.count()}")

# Transform to Silver
df_diagnoses_silver = df_diagnoses_bronze \
    .dropDuplicates(["diagnosis_id"]) \
    .filter(col("diagnosis_id").isNotNull()) \
    .withColumn("severity_score",
        when(col("severity") == "High", 3)
        .when(col("severity") == "Medium", 2)
        .when(col("severity") == "Low", 1)
        .otherwise(0)
    ) \
    .withColumn("severity_level",
        when(col("severity_score") >= 3, "Critical")
        .when(col("severity_score") >= 2, "Moderate")
        .when(col("severity_score") >= 1, "Mild")
        .otherwise("Unknown")
    ) \
    .withColumn("diagnosis_category", substring(col("diagnosis_code"), 1, 3)) \
    .withColumn("diagnosis_description_clean",
        initcap(trim(col("diagnosis_description")))) \
    .withColumn("diagnosis_year", year(col("diagnosis_date"))) \

```



```

.withColumn("diagnosis_month", month(col("diagnosis_date"))) \
.withColumn("is_high_severity", col("severity") == "High") \
.withColumn("is_critical", col("severity_score") >= 3) \
.withColumn("silver_processing_timestamp", current_timestamp()) \
.withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
.drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f" Silver records: {df_diagnoses_silver.count()}")
print(" Severity distribution:")
df_diagnoses_silver.groupBy("severity", "severity_level").count().show()
print(" Sample data:")
display(df_diagnoses_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_diagnoses_path = f"{silver_base_path}diagnoses_silver/"
df_diagnoses_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_diagnoses_path)
print(f" ✓ Saved to: {silver_diagnoses_path}")

# =====
# 1.4 PROCEDURES TABLE - Silver Layer
# =====
print("\n1.4 Processing Procedures Data (Bronze → Silver)...")

# Read from Bronze
bronze_procedures_path = f"{bronze_base_path}procedures/"
df_procedures_bronze = spark.read.format("delta").load(bronze_procedures_path)
print(f" Bronze records: {df_procedures_bronze.count()}")

# Transform to Silver
df_procedures_silver = df_procedures_bronze \
    .dropDuplicates(["procedure_id"]) \
    .filter(col("procedure_id").isNotNull()) \
    .filter(col("cost") > 0) \
    .withColumn("cost_category",
        when(col("cost") < 5000, "Low Cost (<$5K)")
        .when(col("cost") < 15000, "Medium Cost ($5K-$15K)")
        .when(col("cost") < 30000, "High Cost ($15K-$30K)")
        .otherwise("Very High Cost (>$30K)")
    ) \
    .withColumn("cost_bucket",
        when(col("cost") < 10000, "$0-$10K")
        .when(col("cost") < 25000, "$10K-$25K")
        .when(col("cost") < 50000, "$25K-$50K")
        .otherwise("$50K+")
    ) \
    .withColumn("procedure_name_clean", initcap(trim(col("procedure_name")))) \
    .withColumn("procedure_year", year(col("procedure_date"))) \
    .withColumn("procedure_month", month(col("procedure_date"))) \
    .withColumn("is_high_cost", col("cost") > 30000) \
    .withColumn("is_expensive", col("cost") > 50000) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \

```

```

        .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_procedures_silver.count()}")
print(f"  Total cost: ${df_procedures_silver.agg(sum('cost')).collect()[0][0]:,.2f}")
print("  Cost category distribution:")
df_procedures_silver.groupBy("cost_category").count().show()
print("  Sample data:")
display(df_procedures_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_procedures_path = f"{silver_base_path}procedures_silver/"
df_procedures_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_procedures_path)
print(f"  ✓ Saved to: {silver_procedures_path}")

# =====
# 1.5 MEDICATIONS TABLE - Silver Layer
# =====
print("\n1.5 Processing Medications Data (Bronze → Silver)...")

# Read from Bronze
bronze_medications_path = f"{bronze_base_path}medications/"
df_medications_bronze = spark.read.format("delta").load(bronze_medications_path)
print(f"  Bronze records: {df_medications_bronze.count()}")

# Transform to Silver
df_medications_silver = df_medications_bronze \
    .dropDuplicates(["medication_id"]) \
    .filter(col("medication_id").isNotNull()) \
    .withColumn("treatment_duration_days",
        datediff(col("end_date"), col("start_date")))
    ) \
    .withColumn("treatment_duration_category",
        when(col("treatment_duration_days") <= 30, "Short Term (≤30 days)")
        .when(col("treatment_duration_days") <= 90, "Medium Term (31-90 days)")
        .when(col("treatment_duration_days") <= 180, "Long Term (91-180 days)")
        .otherwise("Chronic (>180 days)")
    ) \
    .withColumn("is_active", col("end_date") >= current_date()) \
    .withColumn("has_started", col("start_date") <= current_date()) \
    .withColumn("dosage_amount",
        regexp_extract(col("dosage"), "^[0-9]+", 1).cast(IntegerType())
    ) \
    .withColumn("dosage_unit", regexp_extract(col("dosage"), "[a-zA-Z]+$", 0)) \
    .withColumn("frequency_standardized",
        when(col("frequency").contains("Daily"), "Daily")
        .when(col("frequency").contains("Weekly"), "Weekly")
        .when(col("frequency").contains("Monthly"), "Monthly")
        .otherwise("Other")
    ) \
    .withColumn("medicine_name_clean", initcap(trim(col("medicine_name")))) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \

```

```

        .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_medications_silver.count()}")
print("  Treatment duration distribution:")
df_medications_silver.groupBy("treatment_duration_category").count().show()
print("  Sample data:")
display(df_medications_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_medications_path = f"{silver_base_path}medications_silver/"
df_medications_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_medications_path)
print(f"  ✓ Saved to: {silver_medications_path}")

# =====
# 1.6 DOCTORS TABLE - Silver Layer
# =====
print("\n1.6 Processing Doctors Data (Bronze → Silver)...")

# Read from Bronze
bronze_doctors_path = f"{bronze_base_path}doctors/"
df_doctors_bronze = spark.read.format("delta").load(bronze_doctors_path)
print(f"  Bronze records: {df_doctors_bronze.count()}")

# Transform to Silver
df_doctors_silver = df_doctors_bronze \
    .dropDuplicates(["doctor_id"]) \
    .filter(col("doctor_id").isNotNull()) \
    .withColumn("doctor_name_clean", initcap(trim(col("doctor_name")))) \
    .withColumn("first_name", split(col("doctor_name_clean"), " ")[0]) \
    .withColumn("last_name", split(col("doctor_name_clean"), " ")[1]) \
    .withColumn("specialty_clean", initcap(trim(col("specialty")))) \
    .withColumn("years_of_experience",
        floor(datediff(current_date(), col("hire_date")) / 365.25)
    ) \
    .withColumn("experience_level",
        when(col("years_of_experience") < 5, "Junior (<5 years)")
        .when(col("years_of_experience") < 10, "Mid-Level (5-10 years)")
        .when(col("years_of_experience") < 20, "Senior (10-20 years)")
        .otherwise("Expert (>20 years)")
    ) \
    .withColumn("is_valid_email",
        col("email").rlike("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")
    ) \
    .withColumn("phone_formatted", regexp_replace(col("phone_number"), "[^0-9]", "")) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_doctors_silver.count()}")
print("  Experience level distribution:")
df_doctors_silver.groupBy("experience_level").count().show()
print("  Sample data:")

```



```

display(df_doctors_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_doctors_path = f"{silver_base_path}doctors_silver/"
df_doctors_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_doctors_path)
print(f" ✓ Saved to: {silver_doctors_path}")

# =====
# 1.7 DEPARTMENTS TABLE - Silver Layer
# =====
print("\n1.7 Processing Departments Data (Bronze → Silver)...")

# Read from Bronze
bronze_departments_path = f"{bronze_base_path}departments/"
df_departments_bronze = spark.read.format("delta").load(bronze_departments_path)
print(f" Bronze records: {df_departments_bronze.count()}")

# Transform to Silver
df_departments_silver = df_departments_bronze \
    .dropDuplicates(["department_id"]) \
    .filter(col("department_id").isNotNull()) \
    .withColumn("department_name_clean", initcap(trim(col("department_name")))) \
    .withColumn("manager_name_clean", initcap(trim(col("manager_name")))) \
    .withColumn("manager_first_name", split(col("manager_name_clean"), " ")[0]) \
    .withColumn("manager_last_name", split(col("manager_name_clean"), " ")[1]) \
    .withColumn("floor_category",
        when(col("floor_number") <= 2, "Lower Floor")
        .when(col("floor_number") <= 5, "Middle Floor")
        .otherwise("Upper Floor")
    ) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f" Silver records: {df_departments_silver.count()}")
print(" Sample data:")
display(df_departments_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_departments_path = f"{silver_base_path}departments_silver/"
df_departments_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_departments_path)
print(f" ✓ Saved to: {silver_departments_path}")

# =====
# 1.8 INSURANCE PROVIDERS TABLE - Silver Layer
# =====
print("\n1.8 Processing Insurance Providers Data (Bronze → Silver)...")

# Read from Bronze

```



```

bronze_insurance_path = f"{bronze_base_path}insurance_providers/"
df_insurance_bronze = spark.read.format("delta").load(bronze_insurance_path)
print(f"  Bronze records: {df_insurance_bronze.count()}")

# Transform to Silver
df_insurance_silver = df_insurance_bronze \
    .dropDuplicates(["insurance_provider_id"]) \
    .filter(col("insurance_provider_id").isNotNull()) \
    .withColumn("provider_name_clean", initcap(trim(col("provider_name")))) \
    .withColumn("plan_type_standardized",
        when(upper(col("plan_type")).contains("MEDICARE"), "Medicare")
        .when(upper(col("plan_type")).contains("MEDICAID"), "Medicaid")
        .when(upper(col("plan_type")).contains("HMO"), "HMO")
        .when(upper(col("plan_type")).contains("PPO"), "PPO")
        .when(upper(col("plan_type")).contains("EPO"), "EPO")
        .otherwise("Other")
    ) \
    .withColumn("coverage_level",
        when(col("coverage_percent") >= 90, "Premium (90%+)")
        .when(col("coverage_percent") >= 80, "High (80-89%)")
        .when(col("coverage_percent") >= 70, "Standard (70-79%)")
        .otherwise("Basic (<70%)")
    ) \
    .withColumn("contact_number_formatted", regexp_replace(col("contact_number"), "[^0-9]", "")) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_insurance_silver.count()}")
print("  Coverage level distribution:")
df_insurance_silver.groupBy("coverage_level", "plan_type_standardized").count().show()
print("  Sample data:")
display(df_insurance_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_insurance_path = f"{silver_base_path}insurance_providers_silver/"
df_insurance_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_insurance_path)
print(f"  ✓ Saved to: {silver_insurance_path}")

# =====
# 1.9 APPOINTMENTS TABLE - Silver Layer
# =====
print("\n1.9 Processing Appointments Data (Bronze → Silver)...")

# Read from Bronze
bronze_appointments_path = f"{bronze_base_path}appointments/"
df_appointments_bronze = spark.read.format("delta").load(bronze_appointments_path)
print(f"  Bronze records: {df_appointments_bronze.count()}")

# Transform to Silver
df_appointments_silver = df_appointments_bronze \

```

```

.dropDuplicates(["appointment_id"]) \
.filter(col("appointment_id").isNotNull()) \
.withColumn("is_completed", col("status") == "Completed") \
.withColumn("is_no_show", col("status") == "No Show") \
.withColumn("is_cancelled", col("status") == "Cancelled") \
.withColumn("is_scheduled", col("status") == "Scheduled") \
.withColumn("appointment_hour", split(col("appointment_time"),
":")[0].cast(IntegerType())) \
.withColumn("time_of_day",
    when(col("appointment_hour") < 12, "Morning")
    .when(col("appointment_hour") < 17, "Afternoon")
    .otherwise("Evening")
) \
.withColumn("appointment_year", year(col("appointment_date"))) \
.withColumn("appointment_month", month(col("appointment_date"))) \
.withColumn("appointment_dayofweek", dayofweek(col("appointment_date"))) \
.withColumn("day_of_week",
    when(col("appointment_dayofweek") == 1, "Sunday")
    .when(col("appointment_dayofweek") == 2, "Monday")
    .when(col("appointment_dayofweek") == 3, "Tuesday")
    .when(col("appointment_dayofweek") == 4, "Wednesday")
    .when(col("appointment_dayofweek") == 5, "Thursday")
    .when(col("appointment_dayofweek") == 6, "Friday")
    .otherwise("Saturday")
) \
.withColumn("reason_clean", initcap(trim(col("reason")))) \
.withColumn("is_urgent",
    col("reason_clean").isin("Chest Pain", "Fracture", "COVID-19")
) \
.withColumn("silver_processing_timestamp", current_timestamp()) \
.withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
.drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f" Silver records: {df_appointments_silver.count()}")
print(" Appointment status distribution:")
df_appointments_silver.groupBy("status").count().show()
print(" Sample data:")
display(df_appointments_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_appointments_path = f"{silver_base_path}appointments_silver/"
df_appointments_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_appointments_path)
print(f" ✓ Saved to: {silver_appointments_path}")

# =====
# 1.10 BILLING TABLE - Silver Layer
# =====
print("\n1.10 Processing Billing Data (Bronze → Silver)...")

# Read from Bronze
bronze_billing_path = f"{bronze_base_path}billing/"
df_billing_bronze = spark.read.format("delta").load(bronze_billing_path)

```

```

print(f"   Bronze records: {df_billing_bronze.count()}")

# Transform to Silver
df_billing_silver = df_billing_bronze \
    .dropDuplicates(["bill_id"]) \
    .filter(col("bill_id").isNotNull()) \
    .filter(col("total_amount") > 0) \
    .withColumn("insurance_coverage_percent",
        when(col("total_amount") > 0,
            round((col("insurance_paid") / col("total_amount")) * 100, 2)
        ).otherwise(0)
    ) \
    .withColumn("patient_responsibility_percent",
        when(col("total_amount") > 0,
            round((col("patient_paid") / col("total_amount")) * 100, 2)
        ).otherwise(0)
    ) \
    .withColumn("is_fully_paid", col("payment_status") == "Paid") \
    .withColumn("is_pending", col("payment_status") == "Pending") \
    .withColumn("is_partially_paid", col("payment_status") == "Partially Paid") \
    .withColumn("outstanding_amount",
        col("total_amount") - col("insurance_paid") - col("patient_paid")
    ) \
    .withColumn("is_high_value", col("total_amount") > 100000) \
    .withColumn("bill_amount_category",
        when(col("total_amount") < 25000, "Low (<$25K)")
        .when(col("total_amount") < 50000, "Medium ($25K-$50K)")
        .when(col("total_amount") < 100000, "High ($50K-$100K)")
        .otherwise("Very High (>$100K)")
    ) \
    .withColumn("billing_year", year(col("billing_date"))) \
    .withColumn("billing_month", month(col("billing_date"))) \
    .withColumn("billing_quarter", quarter(col("billing_date"))) \
    .withColumn("payment_efficiency",
        round((col("insurance_paid") + col("patient_paid")) / col("total_amount") * 100,
2)
    ) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"   Silver records: {df_billing_silver.count()}")
print(f"   Total revenue:
${df_billing_silver.agg(sum('total_amount')).collect()[0][0]:.2f}")
print("   Payment status distribution:")
df_billing_silver.groupBy("payment_status").count().show()
print("   Sample data:")
display(df_billing_silver.limit(3))

# Save to Silver Layer (ADLS)
silver_billing_path = f"{silver_base_path}billing_silver/"
df_billing_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_billing_path)

```



```

print(f"  ✓ Saved to: {silver_billing_path}")

print("\n" + "="*60)
print("    SILVER LAYER COMPLETED - All 10 tables saved to ADLS")
print("="*60)

# =====
# SECTION 2: GOLD LAYER TRANSFORMATIONS (Aggregations)
# =====
print("\n" + "="*60)
print("SECTION 2: GOLD LAYER TRANSFORMATIONS (Business Aggregations)")
print("="*60)

# =====
# 2.1 PATIENT 360 AGGREGATION (Gold)
# =====
print("\n2.1 Creating Patient 360 Aggregate View...")

# Join patients with encounters, diagnoses, procedures, billing
df_patients_gold = df_patients_silver \
    .join(df_encounters_silver.groupBy("patient_id").agg(
        count("encounter_id").alias("total_encounters"),
        sum(when(col("is_emergency"), 1).otherwise(0)).alias("emergency_visits"),
        sum(when(col("is_inpatient"), 1).otherwise(0)).alias("inpatient_stays")
    ), "patient_id", "left") \
    .join(df_diagnoses_silver.groupBy("patient_id").agg(
        count("diagnosis_id").alias("total_diagnoses"),
        avg("severity_score").alias("avg_severity_score"),
        count(when(col("severity") == "High", 1)).alias("high_severity_diagnoses")
    ), "patient_id", "left") \
    .join(df_procedures_silver.groupBy("patient_id").agg(
        count("procedure_id").alias("total_procedures"),
        sum("cost").alias("total_procedure_cost"),
        avg("cost").alias("avg_procedure_cost")
    ), "patient_id", "left") \
    .join(df_billing_silver.groupBy("patient_id").agg(
        sum("total_amount").alias("total_billed_amount"),
        sum("insurance_paid").alias("total_insurance_paid"),
        sum("patient_paid").alias("total_patient_paid"),
        avg("insurance_coverage_percent").alias("avg_insurance_coverage")
    ), "patient_id", "left") \
    .fillna(0) \
    .select(
        "patient_id", "full_name", "age", "age_group", "gender_standardized",
        "total_encounters", "emergency_visits", "inpatient_stays",
        "total_diagnoses", "avg_severity_score", "high_severity_diagnoses",
        "total_procedures", "total_procedure_cost", "avg_procedure_cost",
        "total_billed_amount", "total_insurance_paid", "total_patient_paid",
        "avg_insurance_coverage"
    )

print(f"  Gold records: {df_patients_gold.count()} patients")
print("  Sample data:")
display(df_patients_gold.limit(5))

```

```

# Save to Gold Layer (ADLS)
gold_patient360_path = f"{gold_base_path}patient_360/"
df_patients_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_patient360_path)
print(f" ✓ Saved to: {gold_patient360_path}")

# =====
# 2.2 DEPARTMENT PERFORMANCE METRICS (Gold)
# =====
print("\n2.2 Creating Department Performance Metrics...")

df_dept_gold = df_encounters_silver \
    .join(df_departments_silver, df_encounters_silver.department_id ==
df_departments_silver.department_id, "left") \
    .groupBy("department_name_clean", "encounter_type") \
    .agg(
        count("encounter_id").alias("total_encounters"),
        countDistinct("patient_id").alias("unique_patients"),
        countDistinct("doctor_id").alias("unique_doctors"),
        sum(when(col("discharge_status_clean") == "Recovered",
1).otherwise(0)).alias("recovered_count"),
        sum(when(col("discharge_status_clean") == "Deceased",
1).otherwise(0)).alias("deceased_count")
    ) \
    .withColumn("recovery_rate",
        round((col("recovered_count") / col("total_encounters")) * 100, 2)
    ) \
    .orderBy(col("total_encounters").desc())

print(f" Gold records: {df_dept_gold.count()} department metrics")
print(" Sample data:")
display(df_dept_gold.limit(10))

# Save to Gold Layer (ADLS)
gold_dept_path = f"{gold_base_path}department_performance/"
df_dept_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_dept_path)
print(f" ✓ Saved to: {gold_dept_path}")

# =====
# 2.3 FINANCIAL DASHBOARD METRICS (Gold)
# =====
print("\n2.3 Creating Financial Dashboard Metrics...")

df_financial_gold = df_billing_silver \
    .groupBy("payment_status", "bill_amount_category") \
    .agg(
        count("bill_id").alias("bill_count"),
        sum("total_amount").alias("total_amount"),
        sum("insurance_paid").alias("total_insurance_paid"),
        sum("patient_paid").alias("total_patient_paid"),

```

```

        avg("insurance_coverage_percent").alias("avg_insurance_coverage"),
        sum("outstanding_amount").alias("total_outstanding")
    ) \
    .orderBy(col("total_amount").desc())

print(f"  Gold records: {df_financial_gold.count()} financial metrics")
print("  Sample data:")
display(df_financial_gold)

# Save to Gold Layer (ADLS)
gold_financial_path = f"{gold_base_path}financial_metrics/"
df_financial_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_financial_path)
print(f"  ✓ Saved to: {gold_financial_path}")

# =====
# 2.4 TOP DIAGNOSIS ANALYSIS (Gold)
# =====
print("\n2.4 Creating Top Diagnosis Analysis...")

df_diagnosis_gold = df_diagnoses_silver \
    .groupBy("diagnosis_description_clean", "severity", "severity_level") \
    .agg(
        count("diagnosis_id").alias("diagnosis_count"),
        countDistinct("patient_id").alias("affected_patients")
    ) \
    .orderBy(col("diagnosis_count").desc()) \
    .limit(20)

print(f"  Gold records: {df_diagnosis_gold.count()} top diagnoses")
print("  Top 10 diagnoses:")
display(df_diagnosis_gold.limit(10))

# Save to Gold Layer (ADLS)
gold_diagnosis_path = f"{gold_base_path}top_diagnoses/"
df_diagnosis_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_diagnosis_path)
print(f"  ✓ Saved to: {gold_diagnosis_path}")

# =====
# 2.5 APPOINTMENT TREND ANALYSIS (Gold)
# =====
print("\n2.5 Creating Appointment Trend Analysis...")

df_appointment_gold = df_appointments_silver \
    .groupBy("appointment_year", "appointment_month", "time_of_day", "day_of_week") \
    .agg(
        count("appointment_id").alias("total_appointments"),
        sum(when(col("is_no_show"), 1).otherwise(0)).alias("no_shows"),
        sum(when(col("is_completed"), 1).otherwise(0)).alias("completed_appointments")
    ) \

```



```

        .withColumn("no_show_rate",
                    round((col("no_shows") / col("total_appointments")) * 100, 2)
        ) \
        .orderBy("appointment_year", "appointment_month")

print(f" Gold records: {df_appointment_gold.count()} appointment trends")
print(" Sample data:")
display(df_appointment_gold.limit(10))

# Save to Gold Layer (ADLS)
gold_appointment_path = f"{gold_base_path}appointment_trends/"
df_appointment_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_appointment_path)
print(f" ✓ Saved to: {gold_appointment_path}")

# =====
# 2.6 DOCTOR PERFORMANCE METRICS (Gold)
# =====
print("\n2.6 Creating Doctor Performance Metrics...")

df_doctor_gold = df_doctors_silver \
    .join(df_encounters_silver.groupBy("doctor_id").agg(
        count("encounter_id").alias("total_encounters"),
        countDistinct("patient_id").alias("unique_patients_seen")
    ), "doctor_id", "left") \
    .join(df_procedures_silver.groupBy("doctor_id").agg(
        count("procedure_id").alias("total_procedures"),
        sum("cost").alias("total_revenue_generated")
    ), "doctor_id", "left") \
    .fillna(0) \
    .select(
        "doctor_id", "doctor_name_clean", "specialty_clean", "years_of_experience",
        "experience_level", "total_encounters", "unique_patients_seen",
        "total_procedures", "total_revenue_generated"
    ) \
    .orderBy(col("total_revenue_generated").desc())

print(f" Gold records: {df_doctor_gold.count()} doctor metrics")
print(" Top 5 doctors by revenue:")
display(df_doctor_gold.limit(5))

# Save to Gold Layer (ADLS)
gold_doctor_path = f"{gold_base_path}doctor_performance/"
df_doctor_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_doctor_path)
print(f" ✓ Saved to: {gold_doctor_path}")

# =====
# 2.7 MONTHLY REVENUE TREND (Gold)
# =====
print("\n2.7 Creating Monthly Revenue Trend...")

```

```

df_revenue_gold = df_billing_silver \
    .groupBy("billing_year", "billing_month") \
    .agg(
        count("bill_id").alias("total_bills"),
        sum("total_amount").alias("total_revenue"),
        sum("insurance_paid").alias("insurance_paid"),
        sum("patient_paid").alias("patient_paid"),
        avg("payment_efficiency").alias("avg_payment_efficiency"),
        sum("outstanding_amount").alias("total_outstanding")
    ) \
    .withColumn("revenue_growth",
        round((col("total_revenue") - lag("total_revenue").over(
            Window.orderBy("billing_year", "billing_month")
        )) / lag("total_revenue").over(Window.orderBy("billing_year", "billing_month"))) *
100, 2)
    ) \
    .orderBy("billing_year", "billing_month")

print(f" Gold records: {df_revenue_gold.count()} monthly revenue records")
print(" Sample data:")
display(df_revenue_gold.limit(12))

# Save to Gold Layer (ADLS)
gold_revenue_path = f"{gold_base_path}monthly_revenue/"
df_revenue_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_revenue_path)
print(f" ✓ Saved to: {gold_revenue_path}")

# =====
# 2.8 PROCEDURE COST ANALYSIS (Gold)
# =====
print("\n2.8 Creating Procedure Cost Analysis...")

df_procedure_cost_gold = df_procedures_silver \
    .groupBy("procedure_name_clean", "cost_category") \
    .agg(
        count("procedure_id").alias("procedure_count"),
        sum("cost").alias("total_cost"),
        avg("cost").alias("avg_cost"),
        min("cost").alias("min_cost"),
        max("cost").alias("max_cost")
    ) \
    .orderBy(col("total_cost").desc()) \
    .limit(20)

print(f" Gold records: {df_procedure_cost_gold.count()} top procedures by cost")
print(" Top 10 procedures by total cost:")
display(df_procedure_cost_gold.limit(10))

# Save to Gold Layer (ADLS)
gold_procedure_cost_path = f"{gold_base_path}procedure_cost_analysis/"
df_procedure_cost_gold.write.format("delta") \

```

```

        .mode("overwrite") \
        .option("overwriteSchema", "true") \
        .save(gold_procedure_cost_path)
print(f"  ✓ Saved to: {gold_procedure_cost_path}")

# =====
# SECTION 3: SUMMARY REPORT
# =====

print("\n" + "="*70)
print("FINAL SUMMARY - ALL LAYERS COMPLETED")
print("="*70)

print("\n  BRONZE LAYER (Raw Data):")
print(f"  • Patients: {df_patients_bronze.count()} records")
print(f"  • Encounters: {df_encounters_bronze.count()} records")
print(f"  • Diagnoses: {df_diagnoses_bronze.count()} records")
print(f"  • Procedures: {df_procedures_bronze.count()} records")
print(f"  • Medications: {df_medications_bronze.count()} records")
print(f"  • Doctors: {df_doctors_bronze.count()} records")
print(f"  • Departments: {df_departments_bronze.count()} records")
print(f"  • Insurance Providers: {df_insurance_bronze.count()} records")
print(f"  • Appointments: {df_appointments_bronze.count()} records")
print(f"  • Billing: {df_billing_bronze.count()} records")

print("\n  SILVER LAYER (Cleaned Data):")
print(f"  • Patients: {df_patients_silver.count()} records")
print(f"  • Encounters: {df_encounters_silver.count()} records")
print(f"  • Diagnoses: {df_diagnoses_silver.count()} records")
print(f"  • Procedures: {df_procedures_silver.count()} records")
print(f"  • Medications: {df_medications_silver.count()} records")
print(f"  • Doctors: {df_doctors_silver.count()} records")
print(f"  • Departments: {df_departments_silver.count()} records")
print(f"  • Insurance Providers: {df_insurance_silver.count()} records")
print(f"  • Appointments: {df_appointments_silver.count()} records")
print(f"  • Billing: {df_billing_silver.count()} records")

print("\n  GOLD LAYER (Business Aggregations):")
print(f"  • Patient 360: {df_patients_gold.count()} records")
print(f"  • Department Performance: {df_dept_gold.count()} records")
print(f"  • Financial Metrics: {df_financial_gold.count()} records")
print(f"  • Top Diagnoses: {df_diagnosis_gold.count()} records")
print(f"  • Appointment Trends: {df_appointment_gold.count()} records")
print(f"  • Doctor Performance: {df_doctor_gold.count()} records")
print(f"  • Monthly Revenue: {df_revenue_gold.count()} records")
print(f"  • Procedure Cost Analysis: {df_procedure_cost_gold.count()} records")

print("\n" + "="*70)
print("  ALL SILVER AND GOLD LAYER TABLES SAVED TO ADLS SUCCESSFULLY!")
print("="*70)

# List all saved locations
print("\n  SAVE LOCATIONS:")
print("\nSilver Layer Tables (ADLS):")
print(f"  • abfss://silver@{storage_account}.dfs.core.windows.net/patients_silver/")
print(f"  • abfss://silver@{storage_account}.dfs.core.windows.net/encounters_silver/")

```



```

print(f"    • abfss://{silver@{storage_account}.dfs.core.windows.net/diagnoses_silver/")
print(f"    • abfss://{silver@{storage_account}.dfs.core.windows.net/procedures_silver/")
print(f"    • abfss://{silver@{storage_account}.dfs.core.windows.net/medications_silver/")
print(f"    • abfss://{silver@{storage_account}.dfs.core.windows.net/doctors_silver/")
print(f"    • abfss://{silver@{storage_account}.dfs.core.windows.net/departments_silver/")
print(f"    •
abfss://{silver@{storage_account}.dfs.core.windows.net/insurance_providers_silver/")
print(f"    • abfss://{silver@{storage_account}.dfs.core.windows.net/appointments_silver/")
print(f"    • abfss://{silver@{storage_account}.dfs.core.windows.net/billing_silver/")

print("\nGold Layer Tables (ADLS):")
print(f"    • abfss://{gold@{storage_account}.dfs.core.windows.net/patient_360/")
print(f"    • abfss://{gold@{storage_account}.dfs.core.windows.net/department_performance/")
print(f"    • abfss://{gold@{storage_account}.dfs.core.windows.net/financial_metrics/")
print(f"    • abfss://{gold@{storage_account}.dfs.core.windows.net/top_diagnoses/")
print(f"    • abfss://{gold@{storage_account}.dfs.core.windows.net/appointment_trends/")
print(f"    • abfss://{gold@{storage_account}.dfs.core.windows.net/doctor_performance/")
print(f"    • abfss://{gold@{storage_account}.dfs.core.windows.net/monthly_revenue/")
print(f"    •
abfss://{gold@{storage_account}.dfs.core.windows.net/procedure_cost_analysis/")

```

```
%sql
```

```

-- =====
-- REGISTER SILVER TABLES IN UNITY CATALOG
-- =====

```

```

USE CATALOG healthcare_catalog_prod;
CREATE SCHEMA IF NOT EXISTS healthcare_catalog_prod.silver;
CREATE SCHEMA IF NOT EXISTS healthcare_catalog_prod.gold;

```

```
-- Silver Tables
```

```

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_patients
USING DELTA LOCATION
'abfss://{silver@adlshealthcare3000.dfs.core.windows.net/patients_silver/';

```

```

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_encounters
USING DELTA LOCATION
'abfss://{silver@adlshealthcare3000.dfs.core.windows.net/encounters_silver/';

```

```

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_diagnoses
USING DELTA LOCATION
'abfss://{silver@adlshealthcare3000.dfs.core.windows.net/diagnoses_silver/';

```

```

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_procedures
USING DELTA LOCATION
'abfss://{silver@adlshealthcare3000.dfs.core.windows.net/procedures_silver/';

```

```

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_medications

```

```
USING DELTA LOCATION
'abfss://silver@adlshealthcare3000.dfs.core.windows.net/medications_silver/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_doctors
USING DELTA LOCATION
'abfss://silver@adlshealthcare3000.dfs.core.windows.net/doctors_silver/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_departments
USING DELTA LOCATION
'abfss://silver@adlshealthcare3000.dfs.core.windows.net/departments_silver/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_insurance_providers
USING DELTA LOCATION
'abfss://silver@adlshealthcare3000.dfs.core.windows.net/insurance_providers_silver/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_appointments
USING DELTA LOCATION
'abfss://silver@adlshealthcare3000.dfs.core.windows.net/appointments_silver/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.silver.silver_billing
USING DELTA LOCATION
'abfss://silver@adlshealthcare3000.dfs.core.windows.net/billing_silver/';

-- Gold Tables
CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.gold.gold_patient_360
USING DELTA LOCATION 'abfss://gold@adlshealthcare3000.dfs.core.windows.net/patient_360/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.gold.gold_department_performance
USING DELTA LOCATION
'abfss://gold@adlshealthcare3000.dfs.core.windows.net/department_performance/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.gold.gold_financial_metrics
USING DELTA LOCATION
'abfss://gold@adlshealthcare3000.dfs.core.windows.net/financial_metrics/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.gold.gold_top_diagnoses
USING DELTA LOCATION
'abfss://gold@adlshealthcare3000.dfs.core.windows.net/top_diagnoses/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.gold.gold_appointment_trends
USING DELTA LOCATION
'abfss://gold@adlshealthcare3000.dfs.core.windows.net/appointment_trends/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.gold.gold_doctor_performance
USING DELTA LOCATION
'abfss://gold@adlshealthcare3000.dfs.core.windows.net/doctor_performance/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.gold.gold_monthly_revenue
USING DELTA LOCATION
'abfss://gold@adlshealthcare3000.dfs.core.windows.net/monthly_revenue/';

CREATE TABLE IF NOT EXISTS healthcare_catalog_prod.gold.gold_procedure_cost_analysis
USING DELTA LOCATION
'abfss://gold@adlshealthcare3000.dfs.core.windows.net/procedure_cost_analysis/';
```

```
-- Verify all tables
SHOW TABLES IN healthcare_catalog_prod.silver;
SHOW TABLES IN healthcare_catalog_prod.gold;
```

Healthcare Analytics Project - Transformation Summary

Complete Overview of Transformations by Layer

BRONZE LAYER (Raw Data Ingestion)

Purpose: Land raw data exactly as received from source systems

Transformations Applied in Bronze Layer:

Transformation Type	Description	Example
Schema Enforcement	Applied explicit schemas to all CSV files	Defined StructType for each table
Data Type Casting	Converted strings to proper data types	DateType(), DoubleType(), IntegerType()
Metadata Addition	Added tracking columns for lineage	ingestion_timestamp, source_file, ingestion_batch_id
No Data Cleaning	Kept raw data as-is, no deduplication or null handling	Preserved all original values

Code Example:

```
python
df_with_metadata = df \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("patients.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
```

Key Characteristics:

- Immutable data (never modified)
- Complete audit trail
- Source system representation
- Can reprocess from here if needed

SILVER LAYER (Data Cleaning & Transformation)

Purpose: Clean, validate, and enrich data for analytics

Transformations Applied in Silver Layer:

1. PATIENTS TABLE Transformations

Transformation	Description	Business Value
Deduplication	Removed duplicate patient records	Clean patient master data
Age Calculation	Calculated age from date_of_birth	Demographic analysis
Age Grouping	Created age categories (Pediatric, Adult, Geriatric)	Targeted healthcare programs
Full Name	Concatenated first + last name	Patient identification
Gender Standardization	Unified gender values (Male/Female/Other)	Consistent reporting
Email Domain Extraction	Extracted domain from email	Communication analysis
City/State Formatting	Capitalized and trimmed	Geographic analysis
Length of Stay	Calculated admission to discharge days	Operational efficiency

python

```
df_patients_silver = df_patients_bronze \
    .dropDuplicates(["patient_id"]) \
    .withColumn("age", floor(datediff(current_date(), col("date_of_birth")) / 365.25)) \
    .withColumn("age_group", when(col("age") < 18, "Pediatric")...) \
    .withColumn("full_name", concat(initcap(col("first_name")), lit(""), initcap(col("last_name"))))
```

2. ENCOUNTERS TABLE Transformations

Transformation	Description	Business Value
Date Parts Extraction	Year, month, day, quarter, week	Time-series analysis
Encounter Type Flags	Emergency, Inpatient, Outpatient	Resource planning
Season Detection	Winter/Spring/Summer/Fall	Seasonal pattern analysis
Admission Source Clean	Standardized source values	Referral analysis
Discharge Status Clean	Standardized outcome values	Quality metrics

3. DIAGNOSES TABLE Transformations

Transformation	Description	Business Value
Severity Scoring	High=3, Medium=2, Low=1	Risk stratification
Severity Level	Critical/Moderate/Mild	Clinical priority
Diagnosis Category	Extracted code prefix	Disease classification
High Severity Flag	Boolean for High severity	Quick filtering

python

```
df_diagnoses_silver = df_diagnoses_bronze \
    .withColumn("severity_score",
        when(col("severity") == "High", 3)
        .when(col("severity") == "Medium", 2)
        .when(col("severity") == "Low", 1)
        .otherwise(0)
    )
```

4. PROCEDURES TABLE Transformations

Transformation	Description	Business Value
Cost Filtering	Removed negative/zero costs	Financial accuracy
Cost	Low/Medium/High/Very	Budget planning

Categorization	High	
Cost Bucketing	0-0-10K, 10K-10K-25K, etc.	Financial analysis
High Cost Flag	Cost > \$30K	Cost containment
Procedure Name Clean	Capitalized and trimmed	Standardization

5. MEDICATIONS TABLE Transformations

Transformation	Description	Business Value
Treatment Duration	Days between start and end	Adherence tracking
Duration Category	Short/Medium/Long/Chronic	Treatment planning
Active Medication Flag	End date >= today	Current medication list
Dosage Parsing	Extracted amount and unit	Clinical analysis
Frequency Standardization	Daily/Weekly/Monthly	Prescription patterns

6. APPOINTMENTS TABLE Transformations

Transformation	Description	Business Value
Status Flags	Completed/No Show/Cancelled/Scheduled	Operational metrics
Time of Day	Morning/Afternoon/Evening	Resource scheduling
Day of Week	Monday to Sunday	Pattern analysis
Urgent Flag	Chest Pain, Fracture, COVID-19	Priority handling

7. BILLING TABLE Transformations

Transformation	Description	Business Value
Insurance Coverage %	(Insurance Paid / Total) * 100	Payer analysis
Patient Responsibility %	(Patient Paid / Total) * 100	Patient financial burden
Outstanding Amount	Total - Insurance - Patient	Revenue cycle management
Payment Efficiency	Paid % of total	Collection effectiveness
Bill Amount Category	Low/Medium/High/Very High	Revenue analysis
High Value Flag	Total > \$100K	Risk identification

8. DOCTORS TABLE Transformations

Transformation	Description	Business Value
Years of Experience	Current date - hire date	Experience tracking
Experience Level	Junior/Mid-Level/Senior/Expert	Staff planning
Name Parsing	First and last name split	Reporting
Email Validation	Regex pattern check	Data quality
Phone Formatting	Removed non-digits	Standardization

9. DEPARTMENTS TABLE Transformations

Transformation	Description	Business Value
Floor Category	Lower/Middle/Upper	Facility management
Manager Name	First and last name	Organization chart

Parsing	split	
---------	-------	--

10. INSURANCE PROVIDERS Transformations

Transformation	Description	Business Value
Plan Type Standardization	Medicare/Medicaid/HMO/PPO/EP O	Payer mix analysis
Coverage Level	Premium/High/Standard/Basic	Contract analysis

GOLD LAYER (Business Aggregations & Analytics)

Purpose: Create business-ready metrics and KPIs for decision making

Transformations Applied in Gold Layer:

1. PATIENT 360 AGGREGATION

Metric	Calculation	Business Use
Total Encounters	COUNT of encounters per patient	Utilization tracking
Emergency Visits	SUM of emergency flags	Acute care analysis
Inpatient Stays	SUM of inpatient flags	Hospitalization patterns
Total Diagnoses	COUNT of diagnoses	Complexity scoring
Average Severity Score	AVG of severity scores	Risk assessment
High Severity Diagnoses	COUNT of High severity	Clinical risk
Total Procedures	COUNT of procedures	Treatment intensity
Total Procedure Cost	SUM of procedure costs	Cost analysis
Total Billed Amount	SUM of billing amounts	Revenue per patient
Avg Insurance Coverage	AVG coverage percentage	Payer analysis

python

```
df_patients_gold = df_patients_silver \
    .join(df_encounters_silver.groupBy("patient_id").agg(
        count("encounter_id").alias("total_encounters"),
        sum(when(col("is_emergency"), 1).otherwise(0)).alias("emergency_visits")
    ), "patient_id", "left")
```

2. DEPARTMENT PERFORMANCE DASHBOARD

Metric	Calculation	Business Use
Total Encounters	COUNT per department	Volume analysis
Unique Patients	COUNT DISTINCT patient_id	Patient reach
Unique Doctors	COUNT DISTINCT doctor_id	Staff allocation
Recovered Count	SUM of recovered status	Success rates
Deceased Count	SUM of deceased status	Mortality tracking
Recovery Rate	(Recovered / Total) * 100	Quality metric

3. FINANCIAL DASHBOARD METRICS

Metric	Calculation	Business Use
Bill Count	COUNT by payment status	Volume by status
Total Amount	SUM of total_amount	Revenue by category
Total Insurance Paid	SUM of insurance_paid	Payer revenue
Total Patient Paid	SUM of patient_paid	Patient revenue
Total Outstanding	SUM of	Accounts

	outstanding_amount	receivable
Avg Insurance Coverage	AVG coverage percentage	Payer performance

4. TOP DIAGNOSIS ANALYSIS

Metric	Calculation	Business Use
Diagnosis Count	COUNT of diagnosis_id	Disease prevalence
Affected Patients	COUNT DISTINCT patient_id	Population health

5. APPOINTMENT TREND ANALYSIS

Metric	Calculation	Business Use
Total Appointments	COUNT by time period	Volume trends
No Shows	SUM of no-show flags	Patient behavior
No Show Rate	(No Shows / Total) * 100	Operational efficiency
Completed Appointments	SUM of completed flags	Utilization rate

6. DOCTOR PERFORMANCE METRICS

Metric	Calculation	Business Use
Total Encounters	COUNT per doctor	Workload
Unique Patients Seen	COUNT DISTINCT patient_id	Patient panel size
Total Procedures	COUNT of procedures	Procedure volume
Total Revenue Generated	SUM of procedure costs	Financial contribution

7. MONTHLY REVENUE TREND

Metric	Calculation	Business Use
Total Bills	COUNT per month	Transaction volume
Total Revenue	SUM of total_amount	Revenue trends
Revenue Growth	Month-over-month % change	Growth tracking
Avg Payment Efficiency	AVG of payment_efficiency	Collection trends
Total Outstanding	SUM of outstanding_amount	AR aging

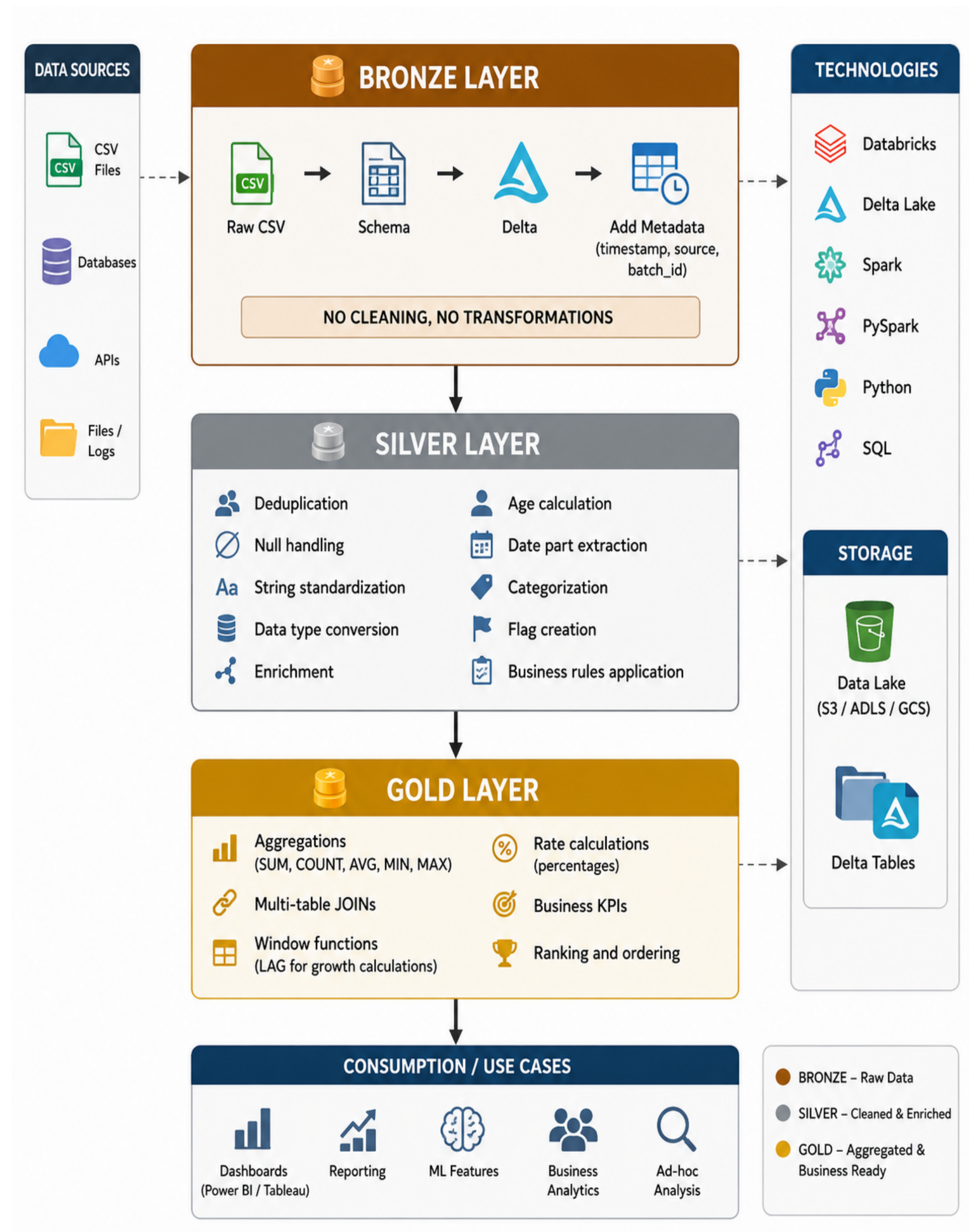
8. PROCEDURE COST ANALYSIS

Metric	Calculation	Business Use
Procedure Count	COUNT per procedure	Utilization
Total Cost	SUM of costs	Cost analysis
Average Cost	AVG of costs	Benchmarking
Min/Max Cost	MIN and MAX values	Cost range

Transformation Summary Table

Layer	Primary Operations	Data Quality	Use Case
Bronze	Schema enforcement, metadata addition, no cleaning	Raw/Untrusted	Audit trail, reprocessing
Silver	Deduplication, null handling, standardization, enrichment, flag creation	Cleaned/Trusted	Analytics, reporting
Gold	Aggregation, joins, calculations, KPIs, business metrics	Curated/Ready	Dashboards, BI, ML

Data Flow with Transformations



Key Business Insights Generated

Gold Table	Key Insights
Patient 360	High-risk patients, frequent visitors, cost drivers
Department Performance	Best performing departments, recovery rates
Financial Metrics	Revenue by payer, collection efficiency, outstanding AR
Top Diagnoses	Most common diseases, population health trends
Appointment Trends	No-show patterns, peak times, seasonal variations
Doctor Performance	Top revenue generators, patient panels, procedure volumes
Monthly Revenue	Growth trends, seasonal patterns, forecasting
Procedure Cost	Most expensive procedures, cost optimization opportunities

This complete transformation pipeline converts raw healthcare data into actionable business intelligence!

